

本地 LLM Agent Prompt/KV 缓存复用的性能评估

元培学院 胡思成 2400017832

摘要

本地 Agent 在长上下文的多轮交互中通常会反复提交系统指令、工具定义、代码摘要或文档片段，而重复的 prefill 会显著推高首 token 延迟（time to first token, TTFT）。为定量分析这一问题，本文测试了 llama.cpp 本地服务中的 prompt/KV 缓存对 coding agent 与 RAG agent 的影响。实验在 Apple M5、32 GB unified memory、Qwen2.5-Coder-3B-Instruct Q4_K_M 与单并发配置下运行，覆盖 2K、4K、8K、16K prompt 长度、三类缓存策略、动态字段位置敏感性以及 `--cache-reuse` 的参数扫描。实验结果说明，**只在服务端打开缓存并不能保证 warm TTFT 改善**；如果动态字段位于 prompt 前部，缓存几乎无法发挥作用。当稳定内容集中在 prompt 前缀中时，Coding 负载在 2K–16K 上获得 **17.0× 到 112.2×** 的 TTFT 加速，RAG 负载获得 **13.6× 到 103.7×** 的加速。在 8K 布局实验中，`front_volatile` 布局会把 warm TTFT 拉回 **9–12 秒量级**，而 `stable_prefix/end_volatile` 布局可以稳定在 **0.15–0.18 秒量级**。当前单会话、稳定前缀设置下，`--cache-reuse` 的影响相对有限；绝大多数组合相对 `reuse=0` 的变化都在约 $\pm 10\%$ 以内。因此，**本地 Agent 的 prompt 组织方式比单纯开启缓存更关键**，稳定、长且可复用的前缀应被视为系统级接口约束。

1 引言

本地 LLM Agent 的一次请求往往不是单句问题，而是由多类上下文共同组成：代码摘要、相关文件、检索文档和历史摘要等内容会被拼接成一个长 prompt。随着交互轮次增加，其中大量内容会跨轮保持不变，实际变化通常集中在当前用户问题、时间戳、request id、最新错误日志或少量对话状态。如果服务端不能复用已经 prefill 的稳定前缀，这些重复上下文仍会在每一轮重新计算，首 token 延迟也会随 prompt 变长而快速上升。

因此，本文主要考察两个问题：prompt/KV 缓存能否把本地单机 Agent 服务中长上下文 warm turn 的 TTFT 降到接近短请求的水平；prompt 又应如何组织，才能稳定触发这种复用。实验部分覆盖关闭缓存、默认缓存、稳定前缀、动态字段位置和 `--cache-reuse` 参数扫描等设置，用于区分“是否启用缓存”和“是否形成稳定可复用前缀”这两个因素。

2 缓存复用机制

对自回归 LLM 服务而言，一轮请求的首 token 延迟主要由三部分组成：新 token 的 prefill 计算、第一个 decode step，以及请求调度、序列化等固定开销。在输出长度固定或较短时，长上下文场景的差异主要来自 prefill。若上一轮已经保留了前缀的 KV 状态，warm turn 不必重新计算被命中的前缀；TTFT 可近似写为

$$T_{\text{TTFT}} \approx T_{\text{prefill}}(L_{\text{new}}) + T_{\text{decode},1} + T_{\text{overhead}},$$

其中 $L_{\text{new}} = L_{\text{prompt}} - L_{\text{reuse}}$ 表示未能复用、仍需重新 prefill 的 token 数。因此，缓存收益不只取决于是否启用缓存，更取决于跨轮 prompt 是否拥有足够长且完全一致的前缀。

后文将关闭缓存、默认缓存、稳定前缀和 `--cache-reuse` 参数扫描分别记为 S0–S3，具体服务端参数和 prompt 组织方式见表 2。

本文以 TTFT 作为核心指标，并同时记录总延迟、解码延迟、解码吞吐、输入 token 数、输出 token 数和服务端常驻内存集大小（resident set size, RSS）。这些指标由流式请求客户端与 `psutil` 采集。为了比较不同缓存配置的收益，加速比（speedup）统一定义为同一负载、同一 prompt 长度下的

$$\text{speedup} = \frac{\text{median TTFT}_{S_0}}{\text{median TTFT}_{\text{strategy}}}.$$

因此，加速比大于 1 表示对应策略比关闭缓存更快。

在 S_0 配置下，服务端关闭 prompt 缓存。即使 prompt 的大部分内容跨轮相同，每一轮也需要重新 prefill 完整上下文。理论上，warm TTFT 会随 prompt 长度近似单调增加；当 prompt 从 2K 扩展到 16K 时，prefill 成本应成为主要瓶颈。

在 S_1 配置下，服务端虽然启用了缓存，但动态字段被放在 prompt 最前部。时间戳、request id 或当前用户请求只要在开头发生变化，后续原本稳定的系统指令、工具定义、代码片段或文档片段就不再构成可命中的公共前缀。此时有效的 L_{reuse} 很小，理论表现应接近 S_0 ；额外的缓存管理还可能带来少量开销。

在 S_2 配置下，稳定内容被集中放在 prompt 前缀，动态字段集中后置。只要系统指令、工具定义、代码或文档上下文和长期摘要保持 token 序列一致，服务端就可以复用大部分 prefill 结果，warm turn 只需处理末尾新增或变化的短后缀。此时 L_{new} 与总 prompt 长度弱相关，TTFT 应接近短请求水平，并且随着上下文变长获得更大的相对加速。

S_3 在 S_2 的稳定前缀基础上扫描 `--cache-reuse`。在单会话、单并发、前缀已经稳定的设置下，主要收益已经由 prompt 组织方式提供，reuse 参数只能影响剩余缓存状态的保留与匹配细节。理论上，它的边际影响应小于 $S_0/S_1/S_2$ 之间的结构性差异；只有在会话切换、上下文接近上限或复用边界频繁变化时，才可能成为主要因素。

3 实验设置

3.1 硬件与软件环境

表 1: 所有正式实验均在同一台本机上顺序运行，并固定为单并发，以减少 slot 调度和并发竞争带来的噪声。

项目	配置
机器	MacBook Pro, Mac17,2
芯片与内存	Apple M5, 10-core CPU, 10-core GPU, 32 GB unified memory
系统与架构	macOS 26.4.1, Darwin arm64
推理服务	llama-server version 9110, Metal enabled, OpenAI 兼容 API
模型	qwen2.5-coder-3b-instruct-q4_k_m.gguf, Qwen2.5-Coder-3B-Instruct, Q4_K_M
Python 与依赖	Python 3.11.14; openai==2.36.0, psutil==7.2.2, pandas==3.0.3, numpy==2.4.4, matplotlib==3.10.9, seaborn==0.13.2
请求设置	temperature=0, top_p=1, max_tokens=64, --parallel 1

3.2 实验负载与缓存策略

实验使用可控的合成负载，在固定任务语义的前提下改变缓存可复用前缀，避免任务内容差异干扰 TTFT 比较。Coding 负载模拟本地代码助手请求，prompt 由系统指令、工具定义、仓库结构、合成源文件、测试文件、对话摘要、动态元数据和当前用户请求组成。RAG 负载模拟长文档问答，prompt 由固定文档上下文和逐轮变化的问题组成。

每个请求序列包含 20 轮请求。统计时将第 1 轮作为 cold turn，正文主要报告第 2–20 轮 warm turn 的中位数。主实验策略见表 2。

表 2: 主实验缓存策略与 prompt 组织方式。

编号	策略名	服务端参数	Prompt 组织方式
S0	no_cache	--no-cache-prompt	使用 stable_prefix 组织, 但服务端关闭复用
S1	default_cache	--cache-prompt	使用 front_volatile 组织, 动态字段位于 prompt 最前部
S2	stable_prefix	--cache-prompt	系统指令、工具定义、代码或文档等稳定内容前置, 动态字段后置
S3	cache-reuse	--cache-prompt	沿用 S2 的组织方式, $N \in \{0, 64, 128, 256, 512\}$
	参数扫描	--cache-reuse N	

4 实验结果

4.1 缓存策略与 TTFT

图 1 的纵轴刻度间距按数值的对数排列, 刻度标签仍为原始数值。



图 1: 主实验结果。

表 3: 主实验 warm-turn 指标。

负载	长度	Warm TTFT (s) ↓			S2 指标		speedup (×) ↑
		S0	S1	S2	总延迟 (s) ↓	RSS (MiB) ↓	
Coding	2K	1.724	1.715	0.101	1.249	2 277.0	17.0
	4K	4.319	4.302	0.126	1.332	2 432.2	34.4
	8K	11.661	11.728	0.181	1.549	2 742.5	64.3
	16K	37.443	37.779	0.334	2.268	3 361.6	112.2
RAG	2K	1.369	1.468	0.108	0.457	2 274.9	13.6
	4K	3.386	3.378	0.128	0.519	2 427.8	26.5
	8K	8.908	8.952	0.173	0.574	2 732.7	51.4
	16K	25.341	26.004	0.244	0.630	3 342.7	103.7

从主实验可以看出, 关闭缓存时, TTFT 基本由长上下文 prefill 主导: Coding 负载从 2K 的 1.72 秒升至 16K 的 37.44 秒, RAG 负载从 1.37 秒升至 25.34 秒。这个增长并不是单纯的固定开销累加。以 Coding 为例, prompt 长度从 4K 增加到 8K 时, warm TTFT 从 4.32 秒升至 11.66 秒; 从 8K 增加到 16K 时, 进一步升至 37.44 秒。RAG 负载也呈现相同趋势, 只是绝对值更低。这说明在当前本地后端和模型配置下, 一旦不能复用前缀, 长 prompt 的 prefill 会迅速成为首 token 延迟的主项, 而不是一个可以被网络或 API 开销掩盖的小项。

在此基础上, S1 用来检验“只开启服务端缓存”是否足够。结果显示, 只把 --cache-prompt 打开, 并不会自然得到 warm-cache 性能。S1 在 2K–16K 上的 TTFT 与 S0 基本重合, Coding 8K 甚至从 11.661 秒小幅变为 11.728 秒, RAG 16K 从 25.341 秒变为 26.004 秒。换言之, 服务端确实被允许保留 prompt cache, 但由于每轮

开头的动态元数据不同，后续长段稳定内容无法成为同一个可命中的公共前缀。这意味着“缓存开关”只是必要条件，“缓存命中条件”才决定性能。

与 S1 不同，S2 的变化来自 prompt 组织方式本身。稳定内容被放在前缀、动态字段被集中后置之后，warm TTFT 不再跟随总 prompt 长度同步上升：Coding 从 2K 的 0.101 秒增至 16K 的 0.334 秒，RAG 从 0.108 秒增至 0.244 秒。也就是说，在 prompt 目标长度扩大 8 倍时，S2 的 TTFT 只增加到约 3.3 倍和 2.3 倍；而 S0 分别增加到约 21.7 倍和 18.5 倍。这个差异说明，S2 并不是降低了每个 token 的 prefill 成本，而是让服务端跳过了大部分已经计算过的稳定前缀，只处理末尾变化的短后缀。因此，随着上下文变长，S0 需要重复计算的部分持续增大，S2 需要新增计算的部分相对稳定，加速比也就从 2K 的 $17.0 \times / 13.6 \times$ 增长到 16K 的 $112.2 \times / 103.7 \times$ 。

从 p95 也可以看到这种收益的稳定性。S2 在 16K Coding 上的 p95 TTFT 为 0.359 秒，16K RAG 为 0.265 秒，均与对应中位数接近；相比之下，S0/S1 的 p95 仍处在几十秒量级。这说明 S2 不是依靠少数几轮偶然命中拉低中位数，而是在第 2–20 轮 warm turn 中持续保持较低 TTFT。对交互式 Agent 来说，因为用户实际感知到的是连续多轮请求中的等待时间，所以稳定的 warm-turn 延迟比单次最快结果更有意义，

总延迟则显示出另一层变化：重复 prefill 被消除后，瓶颈会逐渐转向 decode。以 16K 为例，关闭缓存时，Coding 的中位总延迟为 39.12 秒，其中 37.44 秒已经发生在首 token 之前；RAG 的中位总延迟为 25.78 秒，其中 25.34 秒属于 TTFT。切换到 S2 后，16K Coding 的总延迟降到 2.27 秒，但由于固定输出 64 token，首 token 之后仍需要约 1.93 秒生成；RAG 的回答更短，中位输出约 17–18 token，因此总延迟可以降到 0.63 秒。这意味着，prompt cache 对 TTFT 的改善可以超过百倍，但端到端收益还会受到输出长度影响：短回答场景几乎直接受益于 TTFT 下降，长回答场景则会更早暴露 decode 吞吐这个后续瓶颈。

4.2 动态字段位置敏感性

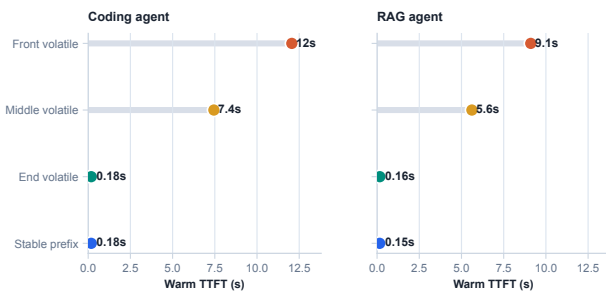


图 2: 8K prompt 下动态字段位置对 warm TTFT 的影响。

表 4: 8K 布局敏感性实验结果。

负载	布局	中位 TTFT(s)	相对 front-volatile 加速(×)
Coding	front_volatile	12.062	1.0
	middle_volatile	7.446	1.6
	end_volatile	0.179	67.3
	stable_prefix	0.180	67.2
RAG	front_volatile	9.104	1.0
	middle_volatile	5.616	1.6
	end_volatile	0.161	56.4
	stable_prefix	0.153	59.4

表 5: 实际 workload 中两类 prompt 组织方式的简化对比。

动态字段前置：破坏前缀复用	稳定内容前置：保留可复用前缀
system = metadata + stable_context user = user_request metadata: current_time, request_id, turn_id, latest_error stable_context: system prompt, tools, repo/doc context	system = stable_context user = metadata + user_request stable_context: system prompt, tools, repo/doc context metadata: current_time, request_id, turn_id, latest_error

图 2 和表 4 将四种 8K prompt 排布放在同一缓存设置下比较，表 5 则给出实验 workload 中对应的构造方式。这里所有配置都打开 `--cache-prompt`，区别只在动态字段的位置。如果 prompt 一开始就是当前时间、trace id、request id 或本轮工具状态，那么这些字段每轮都会变化，后面的系统指令、工具定义和长上下文虽然语义稳定，也不能作为同一段前缀被复用。middle_volatile 的表现介于两者之间：8K Coding 的 TTFT 为 7.446 秒，只比 front_volatile 快 1.6×；RAG 中也约为 1.6×。这说明它确实复用了动态字段之前的一部分内容，但动态字段之后的代码片段、文档段落或检索结果仍然需要重新 prefill，因此收益不会接近完整稳定前缀。

当动态字段被移到末尾后，结果就接近理想的稳定前缀情况。`end_volatile` 和 `stable_prefix` 在 8K Coding 中分别为 0.179 秒和 0.180 秒，在 8K RAG 中分别为 0.161 秒和 0.153 秒，差异已经小到接近运行抖动。相对于 `front_volatile`，这对应 $56\times$ – $67\times$ 的加速。由此可以看出，关键问题不是 prompt 中有没有动态信息，而是这些动态信息是否打断了可复用前缀。只要系统指令、工具 schema、仓库摘要、检索文档和长期记忆摘要能够稳定地排在前面，当前时间、trace id、turn id、最新错误日志、工具返回和当前用户请求就可以放在后部，而不会破坏主要缓存收益。

对实际的 Agent 框架来说，这一结果首先对应到 prompt 的组装规范。许多系统会习惯性地 prompt 开头写入当前时间、会话 id、已用 token 数或随机顺序的 JSON 字段；这些字段通常不是长上下文推理的主体，却会让后续全部稳定内容失去前缀复用机会。相反，把它们移动到用户消息或末尾状态块中，不改变任务信息量，却可以把 warm TTFT 从秒级降到百毫秒级。因此，prompt 布局不应只被看作文案组织方式，而应被视为本地推理服务的性能接口。

4.3 --cache-reuse 参数扫描

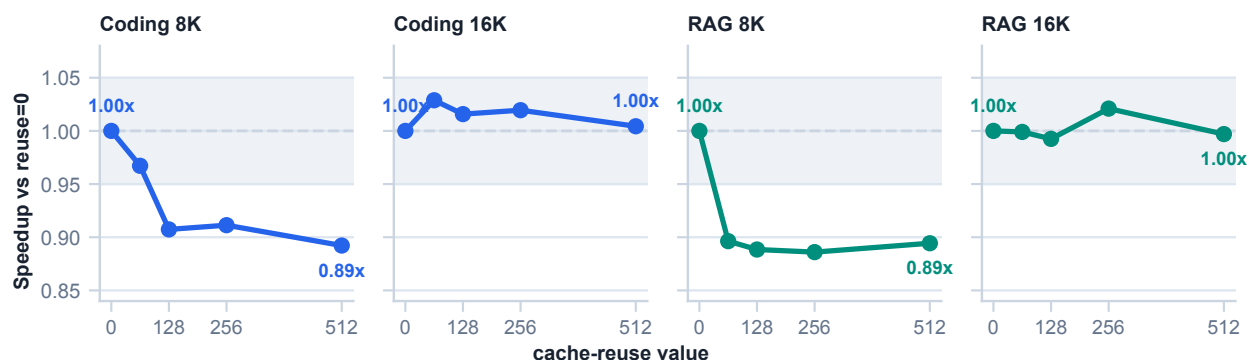


图 3: --cache-reuse 参数扫描。

图 3 以 `reuse=0` 为基准显示不同 `--cache-reuse` 取值下的 TTFT 变化，灰色带对应 $\pm 5\%$ 区间。可以看到，在稳定前缀已经建立之后，`reuse` 参数并没有带来新的数量级差异。8K Coding 中，`reuse=64`、`128`、`256`、`512` 的 TTFT 都慢于 `reuse=0`，其中 `reuse=512` 约慢 10.8%；8K RAG 也类似，非零 `reuse` 大约慢 10%–11%。到 16K 时，曲线进一步靠近基准线：Coding 的最佳点是 `reuse=64`，比 `reuse=0` 快约 2.9%；RAG 的最佳点是 `reuse=256`，比 `reuse=0` 快约 2.1%。

这一结果与前面的布局实验形成对照。对当前单会话、单并发、稳定前缀的请求序列来说，连续两轮请求天然共享同一段长前缀，主要收益已经由 prompt 组织方式获得；`--cache-reuse` 只能影响缓存匹配边界和 KV shifting 的小块复用策略。相比之下，把动态字段从前部移动到末尾可以带来 $50\times$ 以上差距。因此，在类似本文的本地 Agent 设置中，调优顺序应先保证稳定前缀，再考虑 `reuse` 参数；只有在多会话交替、上下文接近上限或 slot 频繁切换时，`--cache-reuse` 才更可能成为需要单独扫描的变量。

4.4 内存占用与吞吐

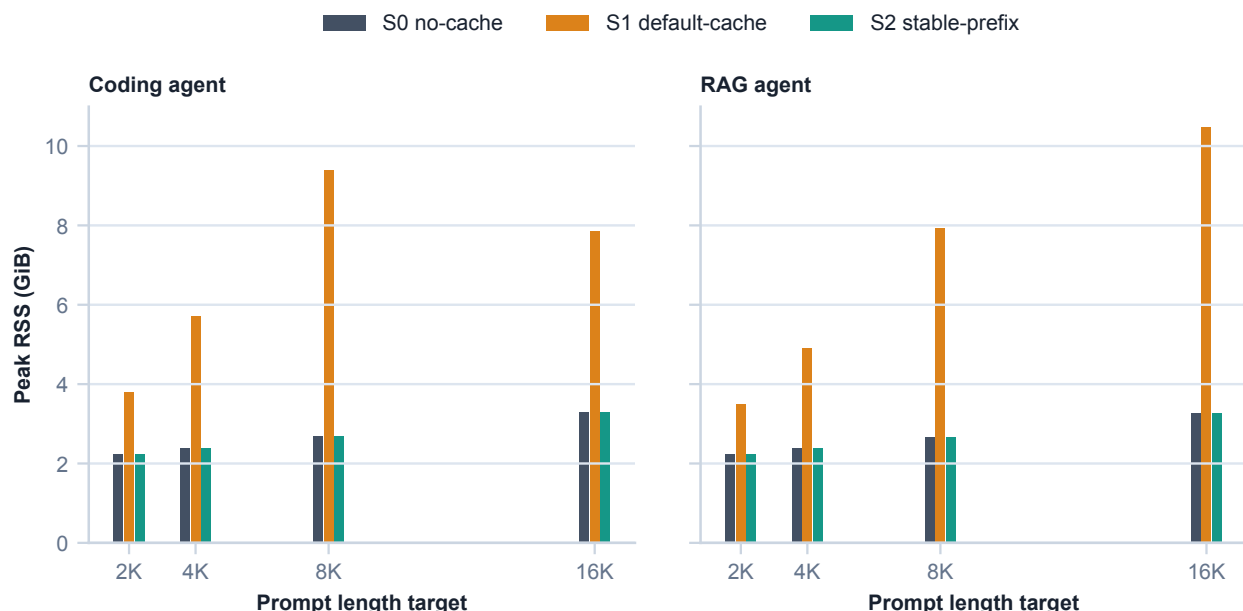


图 4: 不同策略的服务端峰值 RSS。

图 4 显示，在 2K–16K 范围内，3B Q4 模型可以稳定运行在当前 32 GB unified memory 机器上。S2 的峰值 RSS 从约 2.28 GiB 增至约 3.36 GiB，S0 也处在相近水平，说明在本文的单并发设置下，稳定前缀缓存没有带来明显超出上下文长度增长本身的常驻内存压力。相比之下，S1 的内存占用更高：8K Coding 的峰值 RSS 达到 9622.7 MiB，16K RAG 达到 10719.0 MiB，明显高于 S0/S2，但 TTFT 却没有相应改善。这说明缓存状态本身会占用资源；如果 prompt 前缀无法命中，这部分资源开销就很难转化为交互延迟收益。

这里的 RSS 不能直接等同于逐 token KV cache 的理论大小。它还包含模型权重、Metal 后端缓冲区、server 运行时分配和缓存管理状态，因此不同策略之间可能出现非单调波动。对本文关注的问题来说，更有意义的是资源开销和 TTFT 收益是否匹配：S2 在约 3.4 GiB 峰值 RSS 下获得百倍级 TTFT 加速；S1 则消耗更多内存，却仍接近 S0 的延迟水平。由此可以看出，本地 Agent 不能只把缓存视为服务端开关，还需要先保证 prompt 结构能够让缓存命中。

吞吐指标也应放在这个背景下理解。prompt cache 主要减少的是首 token 之前的重复 prefill，并不会直接提高模型逐 token decode 的能力。16K S2 下，Coding 的 decode 吞吐约为 33 token/s，RAG 约为 43 token/s；这些差异更可能来自输出长度、停止位置和运行时抖动，而不是缓存让 decode 阶段本身变快。换言之，S2 的核心价值是让请求更快进入正常生成阶段：短回答场景会直接体现为端到端延迟下降，长回答场景则会在 TTFT 下降后继续受 decode 吞吐限制。因此，TTFT 仍是本文最敏感、也最贴近多轮 Agent 交互体验的指标。

5 结论

在本机 llama.cpp + Qwen2.5-Coder-3B 的实验条件下，prompt/KV 缓存可以把长上下文本地 Agent 的 warm TTFT 从秒级到几十秒级降到百毫秒级，但前提是 prompt 具有长且稳定的可复用前缀。仅打开缓存不足以获得收益；如果动态字段位于 prompt 前部，性能几乎等同于关闭缓存，并且可能增加 RSS。

因此，对本地 coding agent 和 RAG agent 来说，更有效的优化顺序是先建立稳定前缀协议，再考虑 `--cache-reuse`、并发 slot、上下文长度和模型量化等细节参数。系统指令、工具定义、仓库摘要、固定文档上下文和长期记忆摘要等稳定内容应放在 prompt 前部，并保持跨轮字节级稳定；时间戳、request id、

turn id、当前错误日志、工具返回和当前用户请求等动态内容则应集中后置。Agent 框架也应把 prompt 组装视为性能敏感路径，避免在稳定前缀中插入随机顺序的 JSON 字段、当前时间、临时路径、计数器或日志片段。

不过，本文仍存在若干限制。为了隔离变量，实验负载采用合成方式构造，因此无法覆盖真实大型代码库、真实检索系统以及多用户交替会话中的全部复杂性。实验仅在单并发条件下进行，尚未测量多 slot 并发场景中缓存可能发生的抢占或污染。模型方面，本文固定使用 Qwen2.5-Coder-3B Q4_K_M，因此绝对延迟结果不能直接外推到 7B、14B 模型或其他推理后端。尽管如此，布局实验只改变动态字段的位置，变量控制较为明确，因而具有较强的因果解释力，“稳定前缀优先”的判断应当能够迁移到多数支持 prompt 缓存的实现中。